Lab 7: Uninformed Search Algorithms

AI Module
Topic: Semantic Networks, Graphs, and Uninformed Search
Libraries: NetworkX, matplotlib, collections

Learning Objectives

By the end of this lab, you will be able to:

1. **Represent** a problem as a semantic network (graph) using NetworkX.
2. **Implement from scratch** the following uninformed search algorithms:
   - Breadth-First Search (BFS)
   - Depth-First Search (DFS) — both stack-based and recursive
   - Depth-Limited DFS (DLDFS)
   - Iterative Deepening Depth-First Search (IDDFS)
3. **Trace** the traversal order and queue/stack state for each algorithm.
4. **Compare** the algorithms in terms of completeness, optimality, time and space complexity.
5. **Visualise** search behaviour on a graph using matplotlib.
6. **Apply** uninformed search to an implicit state-space (8-puzzle).

Setup

Run the cell below to import the required libraries.

In [ ]:
```
import networkx as nx
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from collections import deque
from copy import deepcopy

# For cleaner notebook output
%matplotlib inline
plt.rcParams['figure.figsize'] = (10, 7)
plt.rcParams['figure.dpi'] = 100
```

Part 1: Building a Semantic Network with NetworkX

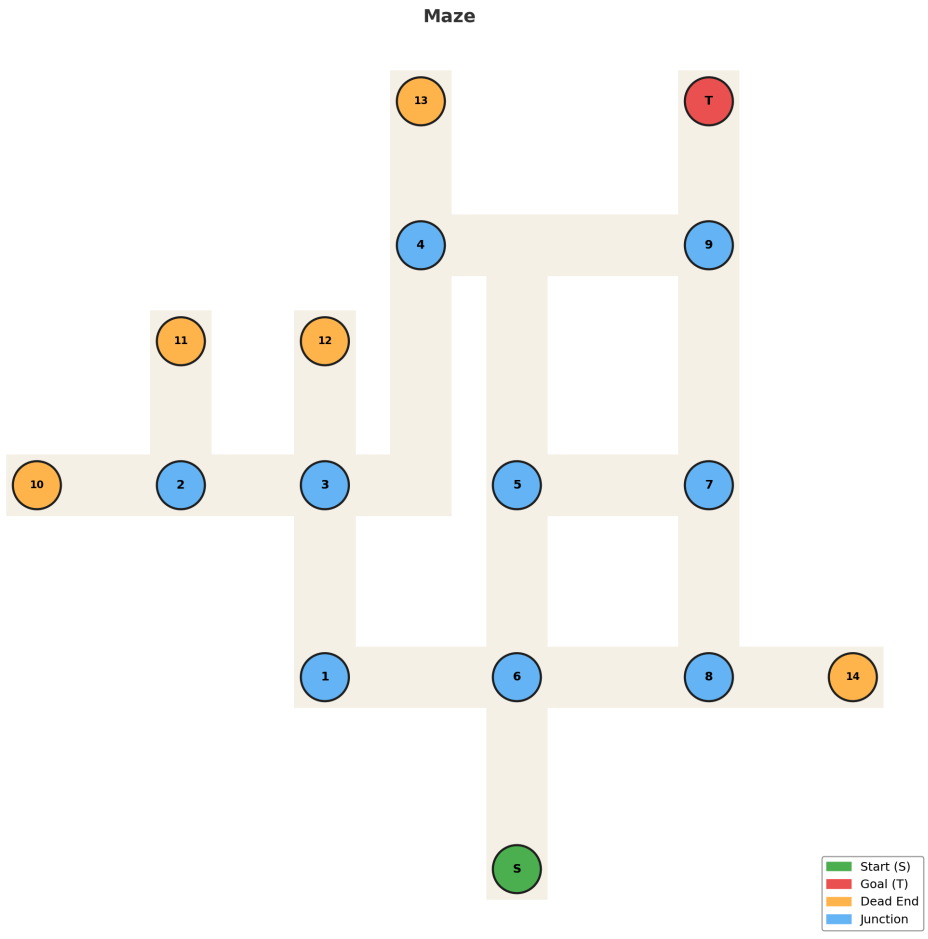
1.1 What is a Semantic Network?

A **semantic network** is a graph consisting of **nodes** (vertices) connected by **links** (edges). Nodes represent objects or states, and edges represent relationships between them. In AI, semantic networks are used to represent a problem's **search space** — the set of all possible states and the transitions between them.

A **semantic tree** is a special case of a semantic network with no cycles. Many search spaces, however, contain cycles (e.g., you can walk from room A to room B and back again). The presence of cycles means our search algorithms must track **visited nodes** to avoid infinite loops.

1.2 The Maze Graph

The figure below illustrates the maze drawing:



The graph below represents this maze. Node **S** is the starting point and node **T** is the exit (goal). The graph is **undirected** (edges go both ways) and contains **cycles** — which makes it a more realistic and interesting search problem than a tree.

The graph has the following structure:

- **S** connects to **1**, **6**, **8**
- **1** connects to **2**, **3**
- **2** connects to **10**, **11** (both dead ends)
- **3** connects to **12** (dead end), **4**
- **4** connects to **13** (dead end), **5**
- **5** connects to **6**, **9**
- **6** connects to **7**
- **7** connects to **8**, **9**
- **8** connects to **14** (dead end)
- **9** connects to **T** (the goal!)

Let's build this graph using NetworkX and visualise it.

In [ ]:
```
# Build the maze graph
maze = nx.Graph()

# Add all edges (undirected — each edge goes both ways)
maze_edges = [
    ('S', '1'), ('S', '6'), ('S', '8'),
    ('1', '2'), ('1', '3'),
    # TODO: add the edges for nodes 2 to 8

    ('9', 'T')
]
maze.add_edges_from(maze_edges)

print(f"Nodes ({maze.number_of_nodes()}): {sorted(maze.nodes())}")
print(f"Edges ({maze.number_of_edges()}): {list(maze.edges())}")
print(f"\nDead-end nodes (degree 1): {[n for n in maze.nodes() if maze.degree(n) == 1]}")
print(f"Cycles present: {len(nx.cycle_basis(maze)) > 0}")
```

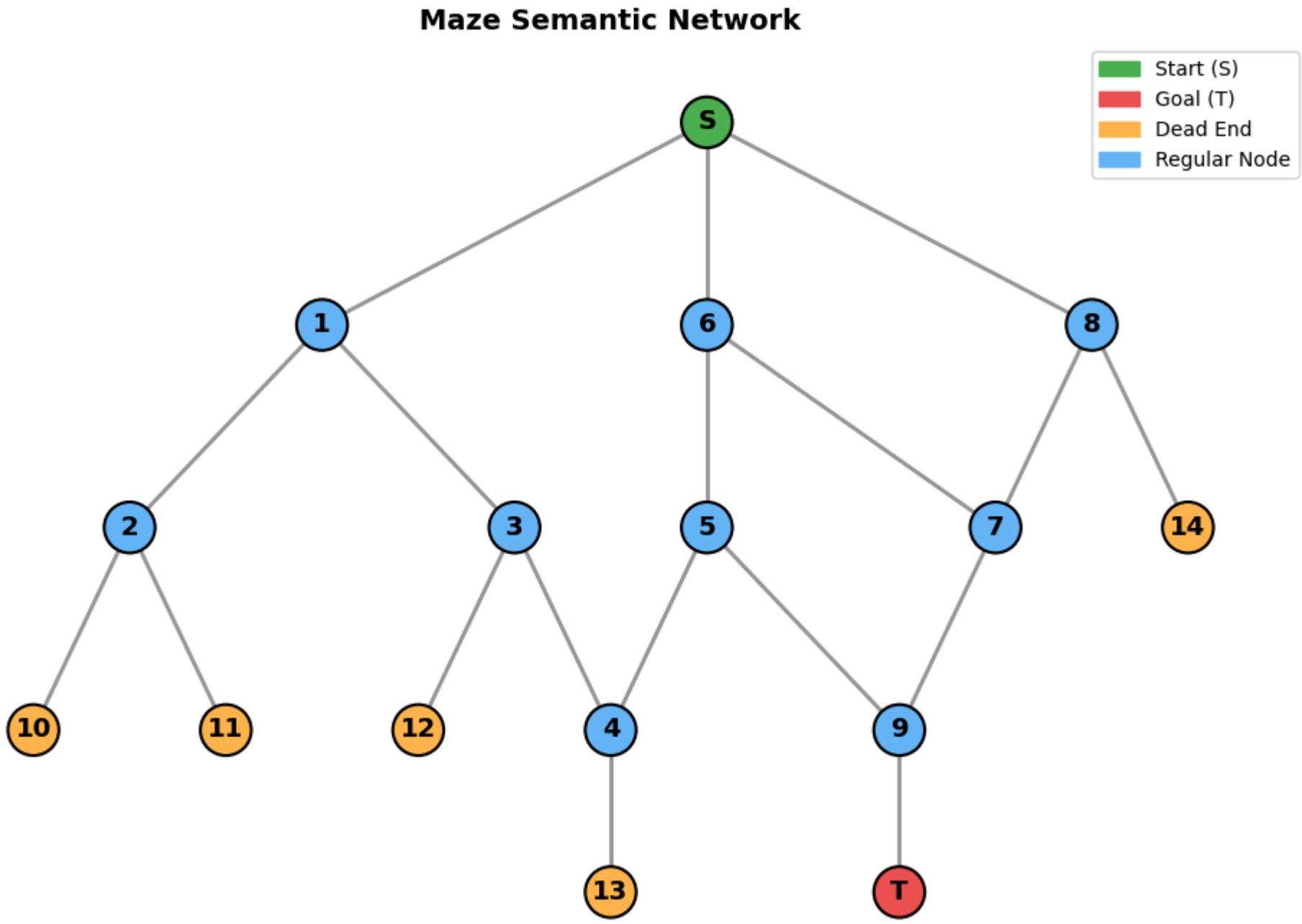
```
Nodes (16): ['1', '10', '11', '12', '13', '14', '2', '3', '4', '5', '6', '7', '8', '9', 'S', 'T']
Edges (18): [('S', '1'), ('S', '6'), ('S', '8'), ('1', '2'), ('1', '3'), ('6', '5'), ('6', '7'), ('8', '7'), ('8', '14'), ('2', '10'), ('2',
'11'), ('3', '12'), ('3', '4'), ('4', '13'), ('4', '5'), ('5', '9'), ('9', '7'), ('9', 'T')]

Dead-end nodes (degree 1): ['10', '11', '12', '13', '14', 'T']
Cycles present: True
```

1.3 Visualising the Graph

A good visualisation helps us understand the structure of the search space. We'll assign fixed positions to each node so that the layout is consistent across all our plots. We'll also colour-code nodes by their role: start (green), goal (red), dead ends (orange), and regular nodes (blue).

In [ ]:
```
# Fixed positions for consistent layout
maze_pos = {
    'S': (0, 3),
    '1': (-2, 2), '6': (0, 2), '8': (2, 2),
    '2': (-3, 1), '3': (-1, 1), '5': (0, 1), '7': (1.5, 1),
    '10': (-3.5, 0), '11': (-2.5, 0), '12': (-1.5, 0), '4': (-0.5, 0),
    '14': (2.5, 1),
    '13': (-0.5, -0.8),
    '9': (1, 0),
    'T': (1, -0.8)
}
```

```python
def get_node_colors(graph, start='S', goal='T'):
    """Return a list of colours for each node based on its role."""
    colors = []
    for n in graph.nodes():
        if n == start:
            colors.append('#4CAF50')   # Green — start
        elif n == goal:
            colors.append('#EF5350')   # Red — goal
        elif graph.degree(n) == 1:
            colors.append('#FFB74D')   # Orange — dead end
        else:
            colors.append('#64B5F6')   # Blue — regular
    return colors

def draw_maze(graph, pos, title='Maze Semantic Network',
              highlight_edges=None, highlight_nodes=None):
    """Draw the maze graph with optional highlighted path."""
    fig, ax = plt.subplots(figsize=(10, 7))

    node_colors = get_node_colors(graph)

    # Draw base graph
    nx.draw(graph, pos, ax=ax, with_labels=True,
            node_color=node_colors, node_size=650,
            font_size=13, font_weight='bold',
            edge_color='#999999', width=2,
            edgecolors='black', linewidths=1.5)

    # Highlight path if provided
    if highlight_edges:
        nx.draw_networkx_edges(graph, pos, edgelist=highlight_edges,
                               edge_color='#E91E63', width=4, ax=ax)
    if highlight_nodes:
        nx.draw_networkx_nodes(graph, pos, nodelist=highlight_nodes,
                               node_color='#E91E63', node_size=650,
                               edgecolors='black', linewidths=1.5, ax=ax)
        nx.draw_networkx_labels(graph, pos,
                                labels={n: n for n in highlight_nodes},
                                font_size=13, font_weight='bold', ax=ax)

    # Legend
    legend_elements = [
        mpatches.Patch(color='#4CAF50', label='Start (S)'),
        mpatches.Patch(color='#EF5350', label='Goal (T)'),
        mpatches.Patch(color='#FFB74D', label='Dead End'),
        mpatches.Patch(color='#64B5F6', label='Regular Node'),
    ]
    if highlight_edges:
        legend_elements.append(mpatches.Patch(color='#E91E63', label='Found Path'))
    ax.legend(handles=legend_elements, loc='upper right', fontsize=10)

    ax.set_title(title, fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()

# Draw the maze
draw_maze(maze, maze_pos)
```



1.4 Understanding the Graph Structure

Before running any search algorithms, let's answer a few questions about the graph.

In [ ]:
```
# Basic graph properties
print("Graph Properties")
print("=" * 40)
# TODO: code to print the number of nodes

# TODO: code to print the number of edges

print(f"Average degree:         {sum(dict(maze.degree()).values()) / maze.number_of_nodes():.2f}")
print(f"Number of cycles:       {len(nx.cycle_basis(maze))}")

print(f"\nAdjacency list:")
for node in sorted(maze.nodes()):
    neighbors = sorted(maze.neighbors(node))
    print(f"  {node:>2} -> {neighbors}")
```

```
Graph Properties
========================================
Number of nodes:        16
Number of edges:        18
Average degree:         2.25
Number of cycles:       3

Adjacency list:
   1 -> ['2', '3', 'S']
  10 -> ['2']
  11 -> ['2']
  12 -> ['3']
  13 -> ['4']
  14 -> ['8']
   2 -> ['1', '10', '11']
   3 -> ['1', '12', '4']
   4 -> ['13', '3', '5']
   5 -> ['4', '6', '9']
   6 -> ['5', '7', 'S']
   7 -> ['6', '8', '9']
   8 -> ['14', '7', 'S']
   9 -> ['5', '7', 'T']
   S -> ['1', '6', '8']
   T -> ['9']
```

> **Key observation:** The average degree of the graph gives us an estimate of the **branching factor** b. The branching factor is crucial because it determines the time complexity ($O(b^d)$) and space complexity of search algorithms, where d is the depth of the goal node.

Part 2: Breadth-First Search (BFS)

2.1 How BFS Works

BFS explores a graph **level by level**. Starting from the initial node, it first visits all nodes at depth 1, then all nodes at depth 2, and so on. It uses a **FIFO queue** (First-In, First-Out) to maintain the frontier of nodes to explore.

Algorithm:

1. Place the starting node in a queue.
2. While the queue is not empty:
   - Remove the node from the **front** of the queue.
   - If it is the goal, return success (and the path).
   - Otherwise, mark it as visited and add its **unvisited neighbours** to the **back** of the queue.

Properties:

- **Complete:** Yes — BFS will always find a goal if one exists (on finite graphs).
- **Optimal:** Yes — BFS finds the **shallowest** goal node, which on an unweighted graph is the shortest path.
- **Time complexity:** $O(b^d)$ where b is the branching factor and d is the depth of the shallowest goal.
- **Space complexity:** $O(b^d)$ — BFS must store all nodes at the current level plus the next level.

2.2 BFS Implementation

We'll implement BFS from scratch using Python's `collections.deque` as our FIFO queue. The function returns both the **traversal order** (the order in which nodes were visited) and the **path** from start to goal.

In [ ]:
```
def bfs(graph, start, goal):
    """
    Breadth-First Search on a NetworkX graph.

    Parameters:
        graph: A NetworkX graph
        start: The starting node
        goal:  The goal node

    Returns:
        traversal_order: List of nodes in the order they were visited
        path:            List of nodes forming the path from start to goal
                         (None if no path exists)
        queue_states:    List of queue snapshots at each step (for tracing)
    """
    visited = set()
    # Each queue entry is (node, path_to_node)
    queue = deque([(start, [start])])
    traversal_order = []
    queue_states = []

    while queue:
        # Record the current state of the queue (for tracing)
        queue_states.append([node for node, _ in queue])

        # Dequeue from the FRONT (FIFO)
        # TODO: use the proper function to dequeue and get 'node' and 'path' from 'queue'


        if node in visited:
            continue

        # TODO: add 'node' to 'visited'

        traversal_order.append(node)

        # Goal test
        if node == goal:
            return traversal_order, path, queue_states

        # Enqueue unvisited neighbours at the BACK
        for neighbor in sorted(graph.neighbors(node)):
            if neighbor not in visited:
                queue.append((neighbor, path + [neighbor]))

    return traversal_order, None, queue_states
```

2.3 Running BFS on the Maze

Let's run BFS on our maze and examine the traversal order and the path found.

In [ ]:
```
bfs_order, bfs_path, bfs_queues = bfs(maze, 'S', 'T')

print("BFS Traversal Order:", bfs_order)
print(f"\nPath found: {' -> '.join(bfs_path)}")
print(f"Path length: {len(bfs_path) - 1} edges")
```

```
BFS Traversal Order: ['S', '1', '6', '8', '2', '3', '5', '7', '14', '10', '11', '12', '4', '9', '13', 'T']

Path found: S -> 6 -> 5 -> 9 -> T
Path length: 4 edges
```

2.4 Tracing the BFS Queue

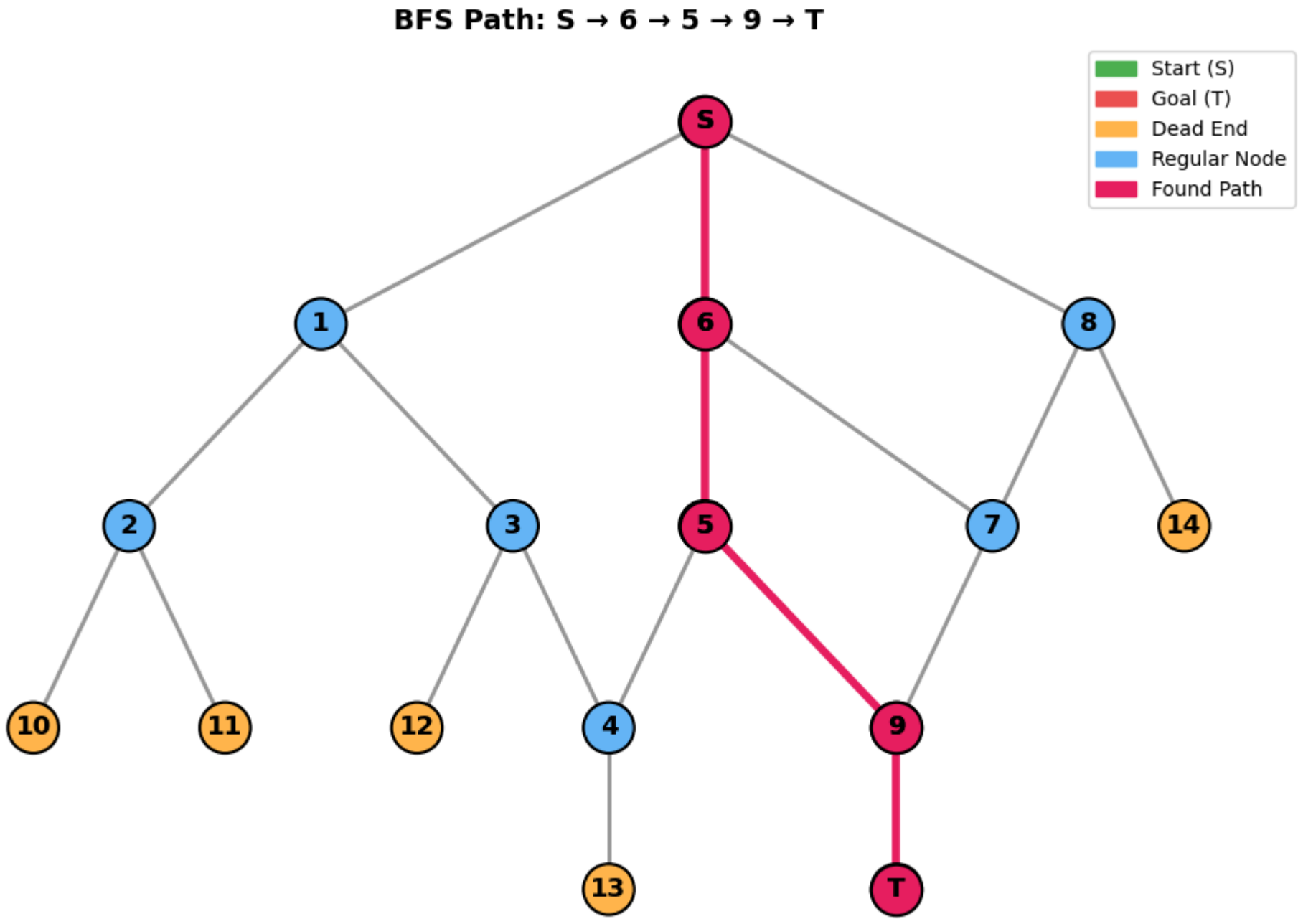
To understand how BFS works, it helps to see the state of the queue at each step.

In [ ]:
```python
def trace_bfs(graph, start, goal):
    """Run BFS and print a detailed trace of the queue at each step."""
    visited = set()
    # TODO: create the queue here (see Cell 2.2)

    step = 0

    print(f"{'Step':>4}  {'Visiting':>10}  {'Queue (after expanding)':}")
    print("-" * 70)

    while queue:
        # TODO: Dequeue node and path from the FRONT (see Cell 2.2)


        if node in visited:
            continue

        # TODO: add 'node' to 'visited'

        step += 1

        if node == goal:
            print(f"{step:>4}  {node:>10}  ** GOAL FOUND **")
            print(f"\nPath: {' -> '.join(path)}")
            print(f"Path length: {len(path) - 1} edges")
            return path

        # Add unvisited neighbours
        for neighbor in sorted(graph.neighbors(node)):
            if neighbor not in visited:
                queue.append((neighbor, path + [neighbor]))

        queue_contents = [n for n, _ in queue]
        print(f"{step:>4}  {node:>10}  {queue_contents}")

    return None

bfs_result = trace_bfs(maze, 'S', 'T')
```

```
Step    Visiting  Queue (after expanding)
----------------------------------------------------------------------
   1           S  ['1', '6', '8']
   2           1  ['6', '8', '2', '3']
   3           6  ['8', '2', '3', '5', '7']
   4           8  ['2', '3', '5', '7', '14', '7']
   5           2  ['3', '5', '7', '14', '7', '10', '11']
   6           3  ['5', '7', '14', '7', '10', '11', '12', '4']
   7           5  ['7', '14', '7', '10', '11', '12', '4', '4', '9']
   8           7  ['14', '7', '10', '11', '12', '4', '4', '9', '9']
   9          14  ['7', '10', '11', '12', '4', '4', '9', '9']
  10          10  ['11', '12', '4', '4', '9', '9']
  11          11  ['12', '4', '4', '9', '9']
  12          12  ['4', '4', '9', '9']
  13           4  ['4', '9', '9', '13']
  14           9  ['9', '13', 'T']
  15          13  ['T']
  16           T  ** GOAL FOUND **

Path: S -> 6 -> 5 -> 9 -> T
Path length: 4 edges
```

2.5 Visualising the BFS Path

In [ ]:
```python
# Convert path to edges for highlighting
bfs_edges = list(zip(bfs_path[:-1], bfs_path[1:]))
draw_maze(maze, maze_pos, title=f'BFS Path: {" → ".join(bfs_path)}',
          highlight_edges=bfs_edges, highlight_nodes=bfs_path)
```



2.6 Verifying with NetworkX

NetworkX has a built-in BFS implementation. Let's verify our result.

In [ ]:
```
# NetworkX BFS shortest path
# TODO: get the shortest path using NetworkX's built-in BFS implementation

print(f"Our BFS path:     {bfs_path}")
print(f"NetworkX BFS path: {list(nx_bfs_path)}")
print(f"Match: {bfs_path == list(nx_bfs_path)}")

# NetworkX BFS traversal order
nx_bfs_order = list(nx.bfs_tree(maze, 'S').nodes())
print(f"\nOur BFS order:     {bfs_order}")
print(f"NetworkX BFS order: {nx_bfs_order}")
```

```
Our BFS path:     ['S', '6', '5', '9', 'T']
NetworkX BFS path: ['S', '6', '5', '9', 'T']
Match: True

Our BFS order:     ['S', '1', '6', '8', '2', '3', '5', '7', '14', '10', '11', '12', '4', '9', '13', 'T']
NetworkX BFS order: ['S', '1', '6', '8', '2', '3', '5', '7', '14', '10', '11', '12', '4', '9', '13', 'T']
```

> **Note:** The NetworkX BFS order may differ slightly from ours if neighbours are iterated in a different order. The *path found* should be the same length (both are optimal), though the exact path may differ if there are multiple shortest paths.

Part 3: Depth-First Search (DFS)

3.1 How DFS Works

DFS explores a graph by going **as deep as possible** along each branch before backtracking. It uses a **LIFO stack** (Last-In, First-Out) — or equivalently, recursion (which uses the call stack).

Key difference from BFS: DFS uses a stack instead of a queue. This means the most recently discovered node is explored first, leading to a depth-first exploration pattern.

Properties:

- **Complete:** Not on infinite graphs or graphs with cycles (unless we track visited nodes). On finite graphs with cycle detection, yes.
- **Optimal:** No — DFS may find a longer path to the goal than necessary.
- **Time complexity:** $O(b^m)$ where m is the maximum depth of the search tree.
- **Space complexity:** $O(b \cdot m)$ — DFS only needs to store the current path and siblings, making it much more memory-efficient than BFS.

3.2 Stack-Based DFS Implementation

In [ ]:
```python
def dfs_stack(graph, start, goal):
    """
    Depth-First Search using an explicit stack.

    Returns:
        traversal_order: List of nodes in the order they were visited
        path:            Path from start to goal (None if not found)
    """
    visited = set()
    # LIFO stack: each entry is (node, path_to_node)
    stack = [(start, [start])]
    traversal_order = []

    while stack:
        # Pop from the TOP (LIFO)
        # TODO: use the proper Python List function to pop and get 'node' and 'path' from 'stack'


        if node in visited:
            continue

        # TODO: add 'node' to 'visited'

        traversal_order.append(node)

        # Goal test
        if node == goal:
            return traversal_order, path

        # Push unvisited neighbours onto the stack
        # We push in REVERSE sorted order so that the alphabetically/numerically
        # first neighbour ends up on TOP of the stack (and gets explored first)
        for neighbor in sorted(graph.neighbors(node), reverse=True):
            if neighbor not in visited:
                stack.append((neighbor, path + [neighbor]))

    return traversal_order, None
```

> **Why reverse order?** When we push neighbours onto a stack, the *last* one pushed is the *first* one popped. By pushing in reverse sorted order, the smallest-valued neighbour ends up on top and gets explored first. This gives us a predictable left-to-right DFS traversal.

3.3 Recursive DFS Implementation

DFS can also be implemented recursively, where the program's **call stack** acts as the implicit LIFO stack.

In [ ]:
```python
def dfs_recursive(graph, start, goal, visited=None, path=None, traversal_order=None):
    """
    Depth-First Search using recursion.

    Returns:
        traversal_order: List of nodes in the order they were visited
        path:            Path from start to goal (None if not found)
    """
    if visited is None:
        visited = set()
        path = [start]
        traversal_order = []

    # TODO: add 'start' to 'visited'

    traversal_order.append(start)

    # Goal test
    if start == goal:
        return traversal_order, path

    # Explore neighbours in sorted order (left-to-right)
    for neighbor in sorted(graph.neighbors(start)):
        if neighbor not in visited:
            result_order, result_path = dfs_recursive(
                graph, neighbor, goal, visited, path + [neighbor], traversal_order
            )
            if result_path is not None:
                return result_order, result_path

    return traversal_order, None
```

3.4 Running DFS on the Maze

In [ ]:
```python
dfs_s_order, dfs_s_path = dfs_stack(maze, 'S', 'T')
# TODO: run the recursive DFS and get the traversal order and path found


print("Stack-based DFS:")
print(f"  Traversal order: {dfs_s_order}")
print(f"  Path found: {' -> '.join(dfs_s_path)}")
print(f"  Path length: {len(dfs_s_path) - 1} edges")
```

```
print(f"\nRecursive DFS:")
print(f"  Traversal order: {dfs_r_order}")
print(f"  Path found: {' -> '.join(dfs_r_path)}")
print(f"  Path length: {len(dfs_r_path) - 1} edges")
```

```
Stack-based DFS:
  Traversal order: ['S', '1', '2', '10', '11', '3', '12', '4', '13', '5', '6', '7', '8', '14', '9', 'T']
  Path found: S -> 1 -> 3 -> 4 -> 5 -> 6 -> 7 -> 9 -> T
  Path length: 8 edges

Recursive DFS:
  Traversal order: ['S', '1', '2', '10', '11', '3', '12', '4', '13', '5', '6', '7', '8', '14', '9', 'T']
  Path found: S -> 1 -> 3 -> 4 -> 5 -> 6 -> 7 -> 9 -> T
  Path length: 8 edges
```

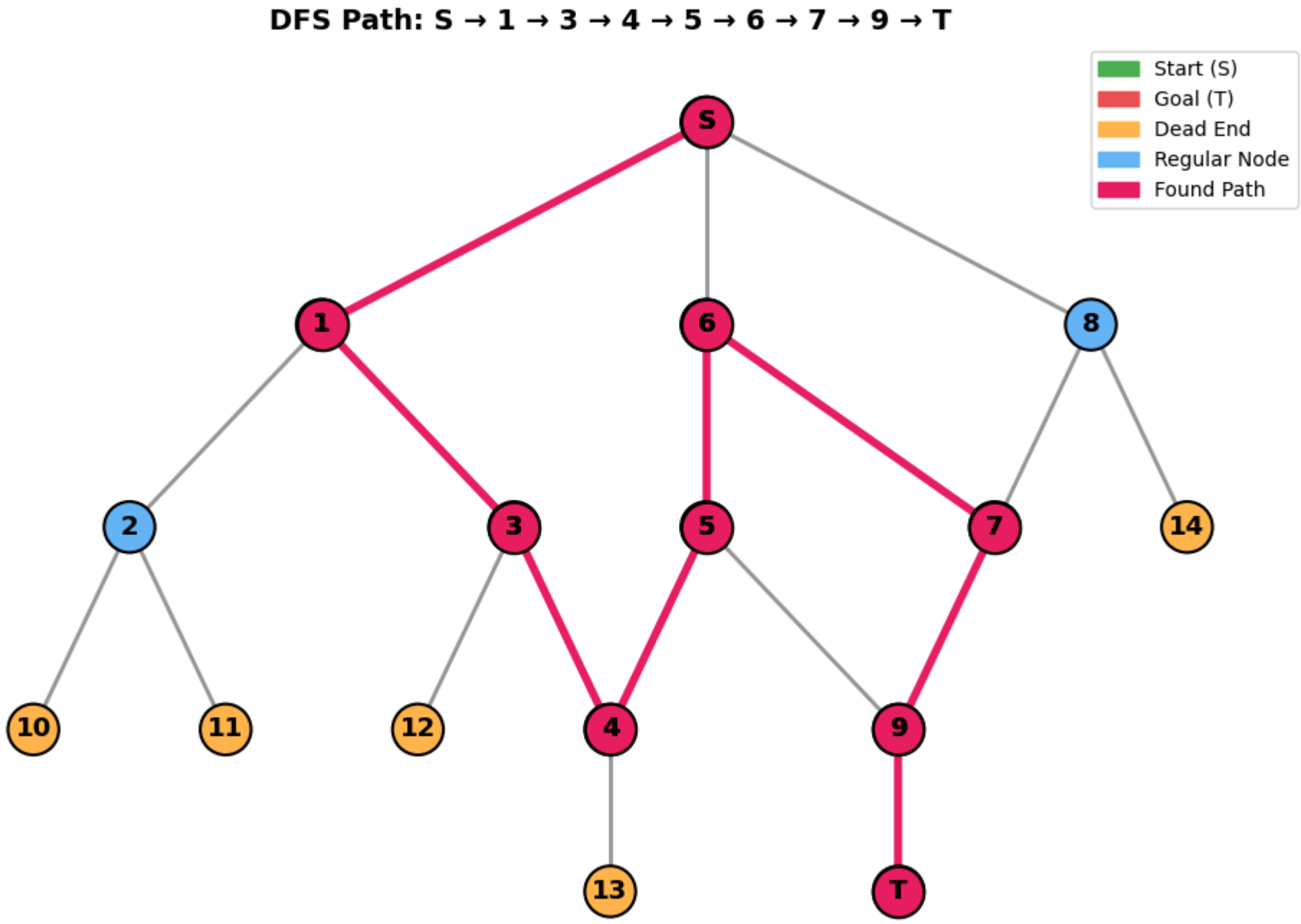
3.5 Tracing DFS (Stack-Based)

In [ ]:
```
def trace_dfs(graph, start, goal):
    """Run stack-based DFS and print a detailed trace."""
    visited = set()
    stack = [(start, [start])]
    step = 0

    print(f"{'Step':>4}  {'Visiting':>10}  {'Stack (after expanding, top on RIGHT)':}")
    print("-" * 70)

    while stack:
        # TODO: pop and get 'node' and 'path' from 'stack' (see Cell 3.2)


        if node in visited:
            continue

        # TODO: add 'node' to 'visited'

        step += 1

        if node == goal:
            print(f"{step:>4}  {node:>10}  ** GOAL FOUND **")
            print(f"\nPath: {' -> '.join(path)}")
            print(f"Path length: {len(path) - 1} edges")
            return path

        for neighbor in sorted(graph.neighbors(node), reverse=True):
            if neighbor not in visited:
                stack.append((neighbor, path + [neighbor]))

        stack_contents = [n for n, _ in stack]
        print(f"{step:>4}  {node:>10}  {stack_contents}")

    return None

dfs_result = trace_dfs(maze, 'S', 'T')
```

```
Step    Visiting  Stack (after expanding, top on RIGHT)
----------------------------------------------------------------------
   1           S  ['8', '6', '1']
   2           1  ['8', '6', '3', '2']
   3           2  ['8', '6', '3', '11', '10']
   4          10  ['8', '6', '3', '11']
   5          11  ['8', '6', '3']
   6           3  ['8', '6', '4', '12']
   7          12  ['8', '6', '4']
   8           4  ['8', '6', '5', '13']
   9          13  ['8', '6', '5']
  10           5  ['8', '6', '9', '6']
  11           6  ['8', '6', '9', '7']
  12           7  ['8', '6', '9', '9', '8']
  13           8  ['8', '6', '9', '9', '14']
  14          14  ['8', '6', '9', '9']
  15           9  ['8', '6', '9', 'T']
  16           T  ** GOAL FOUND **

Path: S -> 1 -> 3 -> 4 -> 5 -> 6 -> 7 -> 9 -> T
Path length: 8 edges
```

3.6 Visualising the DFS Path

In [ ]:
```
dfs_edges = list(zip(dfs_s_path[:-1], dfs_s_path[1:]))
draw_maze(maze, maze_pos, title=f'DFS Path: {" → ".join(dfs_s_path)}',
          highlight_edges=dfs_edges, highlight_nodes=dfs_s_path)
```



3.7 Comparing BFS and DFS

In [ ]:
```
print("Comparison: BFS vs DFS on the Maze")
print("=" * 50)
print(f"{'':>20} {'BFS':>12} {'DFS (stack)':>12}")
print("-" * 50)
print(f"{'Path length':>20} {len(bfs_path)-1:>12} {len(dfs_s_path)-1:>12}")
print(f"{'Nodes visited':>20} {len(bfs_order):>12} {len(dfs_s_order):>12}")
print(f"{'Path':>20} {' -> '.join(bfs_path):>12}")
print(f"{'':>20} {' -> '.join(dfs_s_path):>12}")
print(f"\nBFS found the shorter path? {len(bfs_path) <= len(dfs_s_path)}")
print(f"DFS found the goal faster (fewer nodes visited)? {len(dfs_s_order) <= len(bfs_order)}")
```

```
Comparison: BFS vs DFS on the Maze
==================================================
                              BFS  DFS (stack)
--------------------------------------------------
         Path length            4            8
       Nodes visited           16           16
                Path S -> 6 -> 5 -> 9 -> T
                     S -> 1 -> 3 -> 4 -> 5 -> 6 -> 7 -> 9 -> T

BFS found the shorter path? True
DFS found the goal faster (fewer nodes visited)? True
```

> **Key takeaway:** BFS guarantees the shortest path (in terms of number of edges), but may visit more nodes overall. DFS may find the goal after visiting fewer nodes (if the goal happens to be on the first deep branch it explores), but the path it finds is not guaranteed to be the shortest.

Part 4: Depth-Limited DFS and Iterative Deepening

4.1 The Problem with DFS

DFS can explore very deep paths before finding a goal, especially if the search space has long (or infinite) branches. **Depth-Limited DFS** addresses this by imposing a maximum depth limit on the search.

4.2 Depth-Limited DFS Implementation

In [ ]:
```
def depth_limited_dfs(graph, start, goal, limit):
    """
    Depth-Limited DFS using recursion.

    Parameters:
        graph: A NetworkX graph
        start: The starting node
        goal:  The goal node
```

```
        limit: Maximum depth to search

    Returns:
        traversal_order: Nodes visited in order
        path:            Path to goal, or None
        cutoff_occurred: True if the search was cut off by the depth limit
    """
    visited = set()
    traversal_order = []

    def _dldfs(node, path, depth):
        #TODO: add 'node' to 'visited'

        traversal_order.append(node)

        if node == goal:
            return path, False  # Found goal, no cutoff

        if depth >= limit:
            return None, True   # Reached limit, cutoff occurred

        cutoff = False
        for neighbor in sorted(graph.neighbors(node)):
            if neighbor not in visited:
                result, child_cutoff = _dldfs(neighbor, path + [neighbor], depth + 1)
                if result is not None:
                    return result, False  # Found goal
                if child_cutoff:
                    cutoff = True

        return None, cutoff

    path, cutoff = _dldfs(start, [start], 0)
    return traversal_order, path, cutoff
```

4.3 Testing Depth-Limited DFS at Various Limits

Let's see what happens when we search the maze with different depth limits.

In [ ]:
```
print(f"Depth-Limited DFS on the Maze (S -> T)")
print(f"{'Limit':>6}  {'Found?':>7}  {'Path Length':>12}  {'Nodes Visited':>14}  {'Cutoff?':>8}")
print("-" * 60)

for limit in range(1, 10):
    # TODO: run Depth-Limited DFS and get the resulting 'order', 'path', and 'cutoff'

    found = path is not None
    plen = len(path) - 1 if found else '-'
    print(f"{limit:>6}  {str(found):>7}  {str(plen):>12}  {len(order):>14}  {str(cutoff):>8}")
```

```
Depth-Limited DFS on the Maze (S -> T)
 Limit   Found?   Path Length   Nodes Visited   Cutoff?
------------------------------------------------------------
     1    False             -               4      True
     2    False             -               9      True
     3    False             -              13      True
     4     True             4              16     False
     5    False             -              15      True
     6     True             6              14     False
     7    False             -              14      True
     8     True             8              16     False
     9     True             8              16     False
```

4.4 Iterative Deepening Depth-First Search (IDDFS)

IDDFS combines the completeness and optimality of BFS with the space efficiency of DFS. It works by running Depth-Limited DFS repeatedly with **increasing depth limits** (0, 1, 2, ...) until the goal is found.

Properties:

- **Complete:** Yes.
- **Optimal:** Yes — like BFS, it finds the shallowest goal.
- **Time complexity:** $O(b^d)$ — same as BFS, despite the repeated work.
- **Space complexity:** $O(b \cdot d)$ — same as DFS! This is the key advantage.

> **Why isn't IDDFS wasteful?** Although nodes at shallower depths are re-expanded at each iteration, the vast majority of nodes in a tree are at the deepest level. Mathematically, the overhead factor is only $b/(b-1)$, which approaches a constant (e.g., just 2× for $b = 2$).

4.5 IDDFS Implementation

In [ ]:
```
def iddfs(graph, start, goal, max_depth=50):
    """
    Iterative Deepening Depth-First Search.

    Repeatedly runs depth-limited DFS with increasing limits until the goal is found.

    Returns:
        traversal_order: Nodes visited in the final (successful) iteration
```

```
        path:            Path to goal, or None
        depth_found:     The depth limit at which the goal was found
        total_expanded:  Total nodes expanded across ALL iterations
    """
    total_expanded = 0

    for limit in range(max_depth + 1):
        # TODO: run Depth-Limited DFS and get the resulting 'order', 'path', and 'cutoff'

        total_expanded += len(order)

        if path is not None:
            return order, path, limit, total_expanded

        if not cutoff:
            # No cutoff and no goal — the entire graph has been searched
            break

    return [], None, -1, total_expanded
```

In [ ]:

```
# TODO: run IDDFS and get the resulting order, path, depth, and total nodes


print(f"IDDFS Result:")
print(f"  Goal found at depth limit: {iddfs_depth}")
print(f"  Path: {' -> '.join(iddfs_path)}")
print(f"  Path length: {len(iddfs_path) - 1} edges")
print(f"  Nodes expanded in final iteration: {len(iddfs_order)}")
print(f"  Total nodes expanded across all iterations: {iddfs_total}")
print(f"\nComparison:")
print(f"  BFS path length:   {len(bfs_path) - 1} (nodes expanded: {len(bfs_order)})")
print(f"  IDDFS path length: {len(iddfs_path) - 1} (total expanded: {iddfs_total})")
print(f"  Same optimal path? {len(iddfs_path) == len(bfs_path)}")
```

```
IDDFS Result:
  Goal found at depth limit: 4
  Path: S -> 6 -> 7 -> 9 -> T
  Path length: 4 edges
  Nodes expanded in final iteration: 16
  Total nodes expanded across all iterations: 43

Comparison:
  BFS path length:   4 (nodes expanded: 16)
  IDDFS path length: 4 (total expanded: 43)
  Same optimal path? True
```

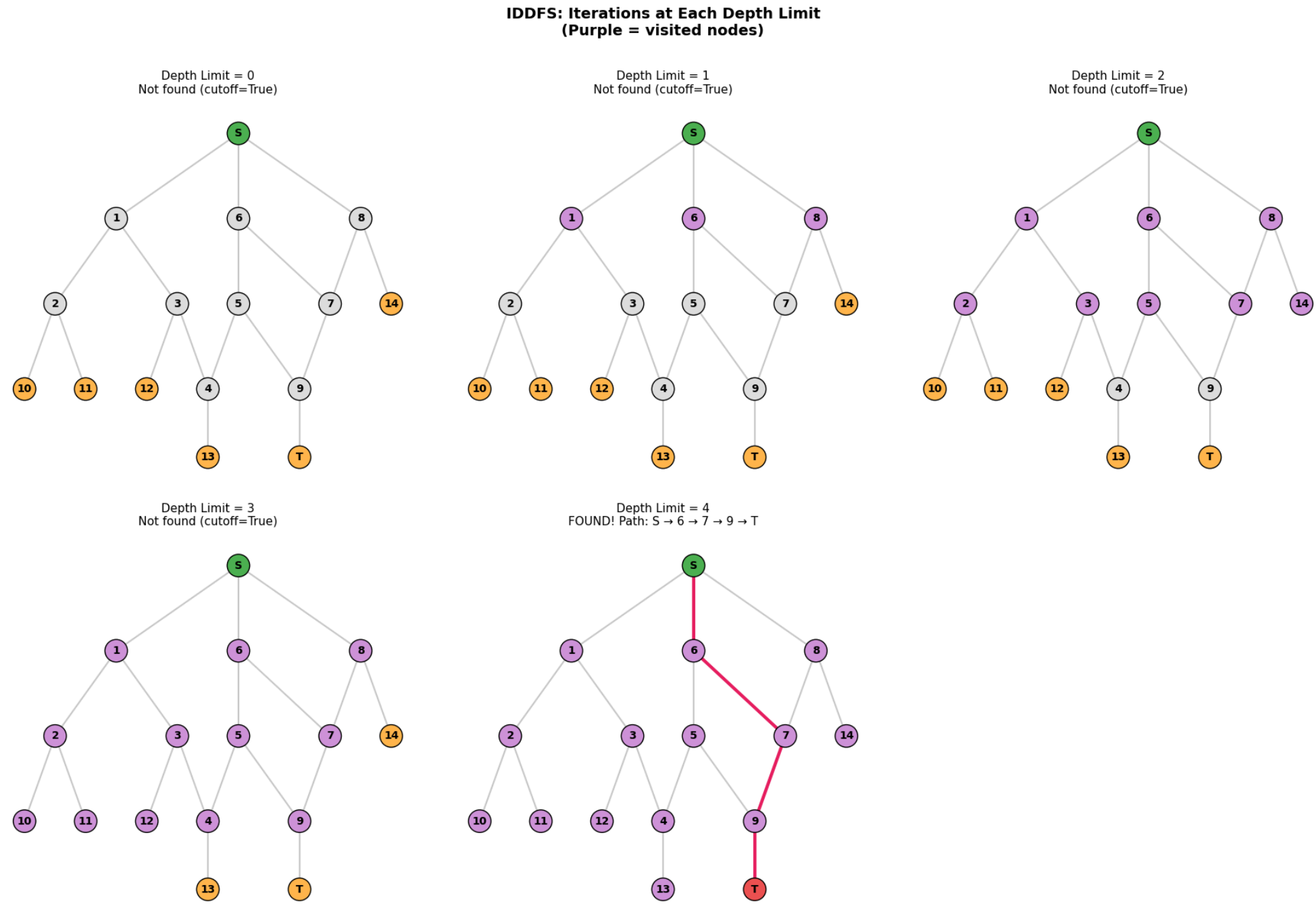
4.6 Visualising the IDDFS Iterations

Let's see what IDDFS explores at each depth limit before finding the goal.

In [ ]:

```
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
axes = axes.flatten()

for i, limit in enumerate(range(iddfs_depth + 1)):
    ax = axes[i]
    # TODO: run Depth-Limited DFS and get the resulting 'order', 'path', and 'cutoff'


    node_colors = []
    for n in maze.nodes():
        if n == 'S':
            node_colors.append('#4CAF50')
        elif n == 'T' and path is not None:
            node_colors.append('#EF5350')
        elif n in order:
            node_colors.append('#CE93D8')   # Purple — visited
        elif maze.degree(n) == 1:
            node_colors.append('#FFB74D')
        else:
            node_colors.append('#E0E0E0')   # Grey — not visited

    nx.draw(maze, maze_pos, ax=ax, with_labels=True,
            node_color=node_colors, node_size=450,
            font_size=10, font_weight='bold',
            edge_color='#CCCCCC', width=1.5,
            edgecolors='black', linewidths=1)

    if path:
        path_edges = list(zip(path[:-1], path[1:]))
        nx.draw_networkx_edges(maze, maze_pos, edgelist=path_edges,
                               edge_color='#E91E63', width=3, ax=ax)

    status = f"FOUND! Path: {' → '.join(path)}" if path else f"Not found (cutoff={cutoff})"
    ax.set_title(f'Depth Limit = {limit}\n{status}', fontsize=11)

# Hide unused subplot
for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

plt.suptitle('IDDFS: Iterations at Each Depth Limit\n(Purple = visited nodes)',
             fontsize=14, fontweight='bold', y=1.01)
```

```
plt.tight_layout()
plt.show()
```



Part 5: Algorithm Comparison Summary

In [ ]:
```
# Comprehensive comparison
print("Algorithm Comparison on the Maze (S -> T)")
print("=" * 75)
print(f"{'Algorithm':<25} {'Path Length':>12} {'Nodes Visited':>14} {'Optimal?':>10} {'Complete?':>10}")
print("-" * 75)

algorithms = [
    ("BFS", bfs_path, bfs_order),
    ("DFS (stack)", dfs_s_path, dfs_s_order),
    # TODO: include recursive DFS results

    # TODO: include IDDFS results

]

for name, path, order in algorithms:
    plen = len(path) - 1 if path else '-'
    optimal = "Yes" if path and len(path) == len(bfs_path) else "No"
    print(f"{name:<25} {str(plen):>12} {len(order):>14} {optimal:>10} {'Yes':>10}")

print(f"\nIDDFS total expanded across all iterations: {iddfs_total}")
```

```
Algorithm Comparison on the Maze (S -> T)
===========================================================================
Algorithm                 Path Length  Nodes Visited   Optimal?  Complete?
---------------------------------------------------------------------------
BFS                                 4             16        Yes        Yes
DFS (stack)                         8             16         No        Yes
DFS (recursive)                     8             16         No        Yes
IDDFS                               4             16        Yes        Yes

IDDFS total expanded across all iterations: 43
```

Properties of Uninformed Search Algorithms

Algorithm	Complete?	Optimal?	Time	Space
BFS	Yes	Yes	$O(b^d)$	$O(b^d)$
DFS	Yes*	No	$O(b^m)$	$O(b \cdot m)$

Algorithm	Complete?	Optimal?	Time	Space
DL-DFS	No (if $\ell < d$)	No	$O(b^\ell)$	$O(b \cdot \ell)$
IDDFS	Yes	Yes	$O(b^d)$	$O(b \cdot d)$

Where: b = branching factor, d = depth of shallowest goal, m = maximum depth, ℓ = depth limit.

* DFS is complete on finite graphs with cycle detection (visited set). Without it, DFS can loop forever on cyclic graphs.

> **IDDFS is often the best uninformed strategy** — it combines the optimality of BFS with the linear space complexity of DFS. The time overhead from repeated iterations is minimal in practice.

Part 6: The 8-Puzzle — State-Space Search

6.1 From Explicit to Implicit Graphs

So far, we've worked with **explicit graphs** — the maze graph was fully defined in memory with all its nodes and edges. Many real AI problems use **implicit graphs** where:

- **States** are generated on-the-fly by applying operators (moves).
- The graph is never fully constructed in memory — only the current frontier is stored.
- The search space can be enormous (the 8-puzzle has 181,440 reachable states).

The **8-puzzle** is a classic AI problem that demonstrates state-space search.

6.2 The 8-Puzzle Problem

The 8-puzzle consists of a 3×3 grid with tiles numbered 1–8 and one blank space. Tiles adjacent to the blank can slide into the blank position. The goal is to reach a specific arrangement from a given starting state.

Goal state:

```
1 | 2 | 3
---------
4 | 5 | 6
---------
7 | 8 |
```

Each state is a node in the implicit graph. Each valid tile move creates an edge to a neighbouring state. The branching factor depends on the blank's position: 2 (corner), 3 (edge), or 4 (centre).

6.3 Representing and Visualising States

In [ ]:
```
def display_state(state, title=None):
    """Display an 8-puzzle state as a grid."""
    if title:
        print(title)
    for i in range(3):
        row = state[i*3:(i+1)*3]
        print(" | ".join(str(x) if x != 0 else " " for x in row))
        if i < 2:
            print("---------")
    print()

def plot_state(state, ax, title=""):
    """Plot an 8-puzzle state as a coloured grid."""
    ax.set_xlim(-0.5, 2.5)
    ax.set_ylim(-0.5, 2.5)
    ax.set_aspect('equal')
    ax.invert_yaxis()
    ax.set_title(title, fontsize=10, fontweight='bold')
    ax.set_xticks([])
    ax.set_yticks([])

    for i in range(3):
        for j in range(3):
            val = state[i * 3 + j]
            color = '#EEEEEE' if val == 0 else '#64B5F6'
            rect = plt.Rectangle((j - 0.45, i - 0.45), 0.9, 0.9,
                                 facecolor=color, edgecolor='black', linewidth=2)
            ax.add_patch(rect)
            if val != 0:
                ax.text(j, i, str(val), ha='center', va='center',
                        fontsize=18, fontweight='bold')

# Define states
goal_state = (1, 2, 3, 4, 5, 6, 7, 8, 0)
start_state = (1, 2, 3, 4, 0, 5, 7, 8, 6)  # 3 moves from goal
```

```
display_state(start_state, "Start State:")
display_state(goal_state, "Goal State:")
```

```
Start State:
1 | 2 | 3
---------
4 |   | 5
---------
7 | 8 | 6

Goal State:
1 | 2 | 3
---------
4 | 5 | 6
---------
7 | 8 |
```

6.4 Generating Successor States

Instead of storing the full graph, we write a function that generates all valid successor states from a given state. This is the key to **implicit graph search**.

In [ ]:
```
def get_successors(state):
    """
    Generate all states reachable from the given 8-puzzle state by one move.

    The blank (0) can swap with any orthogonally adjacent tile.

    Parameters:
        state: A tuple of 9 integers representing the puzzle

    Returns:
        List of (new_state, move_description) tuples
    """
    state = list(state)
    blank = state.index(0)
    row, col = blank // 3, blank % 3

    successors = []
    moves = [(-1, 0, "up"), (1, 0, "down"), (0, -1, "left"), (0, 1, "right")]

    for dr, dc, direction in moves:
        nr, nc = row + dr, col + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            new_state = state[:]
            swap_pos = nr * 3 + nc
            new_state[blank], new_state[swap_pos] = new_state[swap_pos], new_state[blank]
            successors.append((tuple(new_state), f"slide {state[swap_pos]} {direction}"))

    return successors

# Show successors of the start state
print("Successors of the start state:")
display_state(start_state, "Current state:")
for succ, move in get_successors(start_state):
    display_state(succ, f"After: {move}")
```

```
Successors of the start state:
Current state:
1 | 2 | 3
---------
4 |   | 5
---------
7 | 8 | 6

After: slide 2 up
1 |   | 3
---------
4 | 2 | 5
---------
7 | 8 | 6

After: slide 8 down
1 | 2 | 3
---------
4 | 8 | 5
---------
7 |   | 6

After: slide 4 left
1 | 2 | 3
---------
  | 4 | 5
---------
7 | 8 | 6

After: slide 5 right
1 | 2 | 3
---------
4 | 5 |
---------
7 | 8 | 6
```

6.5 BFS on the 8-Puzzle

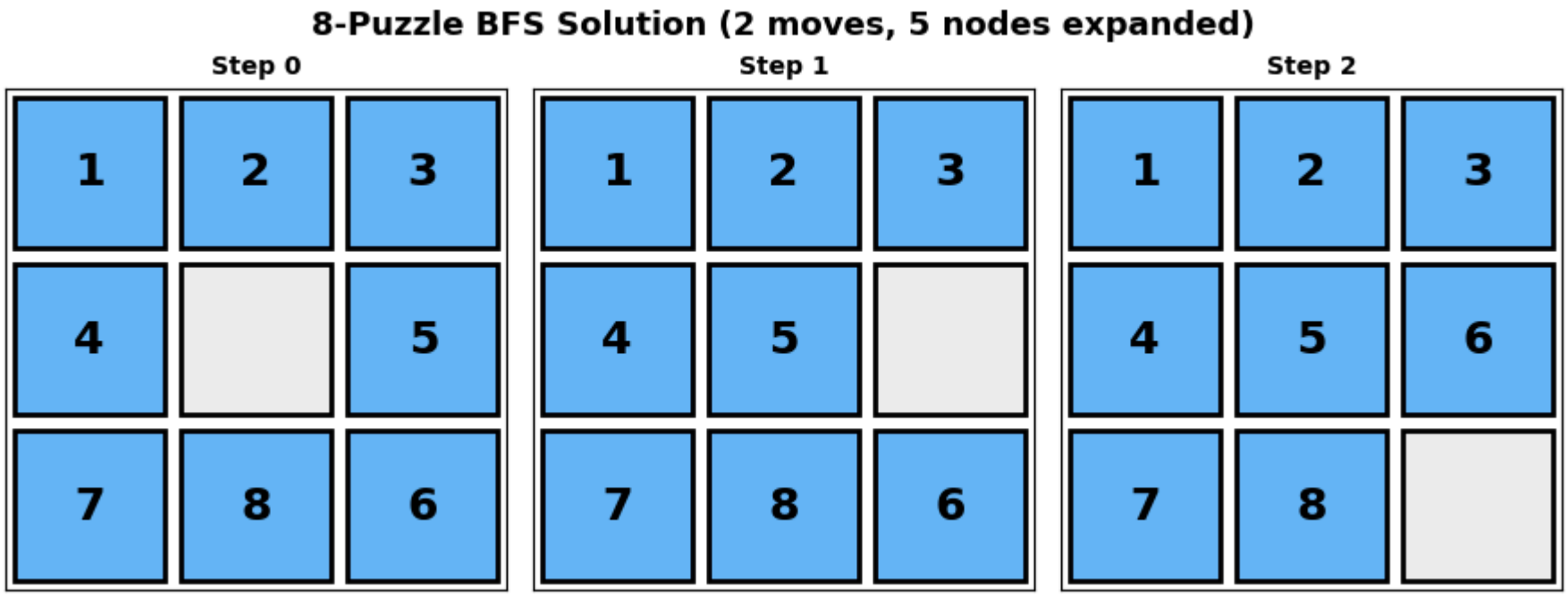
Now we apply our BFS algorithm to the 8-puzzle. Notice that we don't need a NetworkX graph — we generate neighbours on-the-fly using `get_successors`. This is implicit graph search.

In [ ]:
```python
def bfs_8puzzle(start, goal):
    """
    BFS on the 8-puzzle state space (implicit graph).

    Returns:
        path:     List of states from start to goal
        expanded: Number of nodes expanded
    """
    if start == goal:
        return [start], 0

    visited = {start}

    # TODO: create the queue here (see Cell 2.2)

    expanded = 0

    while queue:
        state, path = queue.popleft()
        expanded += 1

        for successor, move in get_successors(state):
            if successor == goal:
                return path + [successor], expanded
            if successor not in visited:
                # TODO: add 'successor' to 'visited'

                queue.append((successor, path + [successor]))

    return None, expanded

# Solve from the start state
# TODO: run BFS and get the resulting path and expanded nodes count

print(f"BFS found a solution in {len(path_8p) - 1} moves, expanding {expanded_8p} nodes.")
print(f"\nSolution sequence:")
for i, state in enumerate(path_8p):
    display_state(state, f"Step {i}:")
```

```
BFS found a solution in 2 moves, expanding 5 nodes.

Solution sequence:
Step 0:
1 | 2 | 3
---------
4 |   | 5
---------
7 | 8 | 6

Step 1:
1 | 2 | 3
---------
4 | 5 |
---------
7 | 8 | 6

Step 2:
1 | 2 | 3
---------
4 | 5 | 6
---------
7 | 8 |
```

6.6 Visualising the 8-Puzzle Solution

In [ ]:
```python
n_steps = len(path_8p)
fig, axes = plt.subplots(1, n_steps, figsize=(3 * n_steps, 3.5))

if n_steps == 1:
    axes = [axes]

for i, (state, ax) in enumerate(zip(path_8p, axes)):
    plot_state(state, ax, title=f"Step {i}")

plt.suptitle(f'8-Puzzle BFS Solution ({n_steps - 1} moves, {expanded_8p} nodes expanded)',
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



6.7 BFS vs DFS on the 8-Puzzle

Let's compare BFS and DFS (with a depth limit) on a slightly harder starting state. This demonstrates why BFS is preferred for finding optimal solutions, while DFS is more memory-efficient.

In [ ]:
```python
# A harder start state (6 moves from goal)
hard_start = (4, 1, 3, 7, 2, 5, 0, 8, 6)
display_state(hard_start, "Harder start state:")

# BFS
# TODO: run BFS and get the resulting path and expanded nodes count

print(f"BFS: {len(bfs_path_hard)-1} moves, {bfs_exp_hard} nodes expanded")

# DFS with depth limit
def dfs_8puzzle(start, goal, max_depth=15):
    """DFS with depth limit for the 8-puzzle."""
    visited = set()
    # TODO: Create the stack here (see Cell 3.5)

    expanded = 0

    while stack:
        # TODO: pop and get 'state' and 'path' from 'stack' (see Cell 3.5)

        if state in visited:
            continue
        # TODO: add 'state' to 'visited'
```

```
        expanded += 1

        if state == goal:
            return path, expanded

        if len(path) - 1 < max_depth:
            for successor, move in get_successors(state):
                if successor not in visited:
                    stack.append((successor, path + [successor]))

    return None, expanded

# TODO: run BFS and get the resulting path and expanded nodes count

dfs_found = dfs_path_hard is not None

print(f"DFS (limit=15): {len(dfs_path_hard)-1 if dfs_found else 'NOT FOUND'} moves, "
      f"{dfs_exp_hard} nodes expanded")

# IDDFS
def iddfs_8puzzle(start, goal, max_depth=20):
    """IDDFS for the 8-puzzle."""
    total_expanded = 0
    for limit in range(max_depth + 1):
        visited = set()
        # TODO: Create the stack here (see Cell 6.7)

        expanded = 0
        found = None
        while stack:
            # TODO: pop and get 'state' and 'path' from 'stack' (see Cell 6.7)

            if state in visited:
                continue
            # TODO: add 'state' to 'visited'

            expanded += 1
            if state == goal:
                found = path
                break
            if len(path) - 1 < limit:
                for successor, move in get_successors(state):
                    if successor not in visited:
                        stack.append((successor, path + [successor]))
        total_expanded += expanded
        if found:
            return found, total_expanded, limit
    return None, total_expanded, -1

# TODO: run IDDFS and get the resulting path and expanded nodes count


print(f"IDDFS: {len(iddfs_path_hard)-1 if iddfs_path_hard else 'NOT FOUND'} moves, "
      f"{iddfs_exp_hard} total nodes expanded (found at depth {iddfs_d_hard})")

print(f"\nComparison:")
print(f"  BFS is optimal ({len(bfs_path_hard)-1} moves) but expanded {bfs_exp_hard} nodes")
if dfs_found:
    print(f"  DFS found a {len(dfs_path_hard)-1}-move solution but expanded {dfs_exp_hard} nodes")
else:
    print(f"  DFS failed to find a solution within the depth limit (expanded {dfs_exp_hard} nodes)")
  print(f"  IDDFS is optimal ({len(iddfs_path_hard)-1} moves) and expanded {iddfs_exp_hard} total nodes")
```

```
Harder start state:
4 | 1 | 3
---------
7 | 2 | 5
---------
  | 8 | 6

BFS: 6 moves, 34 nodes expanded
DFS (limit=15): NOT FOUND moves, 1524 nodes expanded
IDDFS: 6 moves, 191 total nodes expanded (found at depth 6)

Comparison:
  BFS is optimal (6 moves) but expanded 34 nodes
  DFS failed to find a solution within the depth limit (expanded 1524 nodes)
  IDDFS is optimal (6 moves) and expanded 191 total nodes
```

6.8 The Branching Factor of the 8-Puzzle

The branching factor varies depending on where the blank is located.

In [ ]:
```
print("8-Puzzle Branching Factor by Blank Position:")
print()
for pos in range(9):
    state = list(goal_state)
    # Swap blank to this position
    blank_at = state.index(0)
    state[blank_at], state[pos] = state[pos], state[blank_at]
    state = tuple(state)
```

```python
    n_successors = len(get_successors(state))
    row, col = pos // 3, pos % 3
    location = "corner" if (row in [0,2] and col in [0,2]) else (
               "centre" if (row == 1 and col == 1) else "edge")
    print(f"  Blank at position ({row},{col}) [{location}]: {n_successors} successors")

# Compute average branching factor across all blank positions
total_b = 0
for pos in range(9):
    state = list(range(1, 9)) + [0]  # goal state as a list
    blank_idx = state.index(0)       # blank is at position 8
    state[blank_idx], state[pos] = state[pos], state[blank_idx]  # move blank to pos
    total_b += len(get_successors(tuple(state)))
print(f"\nAverage branching factor: {total_b / 9:.2f}  (= {total_b}/9)")
```

```
8-Puzzle Branching Factor by Blank Position:

  Blank at position (0,0) [corner]: 2 successors
  Blank at position (0,1) [edge]: 3 successors
  Blank at position (0,2) [corner]: 2 successors
  Blank at position (1,0) [edge]: 3 successors
  Blank at position (1,1) [centre]: 4 successors
  Blank at position (1,2) [edge]: 3 successors
  Blank at position (2,0) [corner]: 2 successors
  Blank at position (2,1) [edge]: 3 successors
  Blank at position (2,2) [corner]: 2 successors

Average branching factor: 2.67  (= 24/9)
```

6.9 Complexity Implications

With an average branching factor of roughly 2.67 and typical solution depths of around 20 moves for random starts:

Algorithm	Estimated Nodes	Feasible?
BFS	$2.67^{20} \approx 2.3 \times 10^8$	Borderline
DFS (limit=20)	Could explore entire state space	Risky
IDDFS	$\sim 2.67^{20} \times \frac{2.67}{1.67} \approx 3.7 \times 10^8$	Borderline

For harder puzzles, we need **heuristic search** (A*, which we'll cover in Lab 8) to make the problem tractable. The 8-puzzle is a good example of why informed search matters.

Summary

In this lab, we:

1. **Built** a semantic network (graph) using NetworkX to represent a maze.
2. **Implemented** BFS, DFS (stack and recursive), Depth-Limited DFS, and IDDFS **from scratch**.
3. **Traced** the queue/stack state at each step for both BFS and DFS.
4. **Compared** the algorithms on the maze: BFS and IDDFS find optimal paths; DFS does not.
5. **Applied** BFS, DFS, and IDDFS to the 8-puzzle as an example of **implicit state-space search**.
6. **Analysed** branching factor and its impact on algorithm complexity.

Key takeaways:

- BFS is **optimal and complete** but has high space complexity $O(b^d)$.
- DFS is **space-efficient** $O(b \cdot d)$ but not optimal.
- IDDFS gives us the **best of both worlds**: optimal, complete, and space-efficient.
- For large or complex search spaces, uninformed search becomes impractical — motivating the need for **heuristic search algorithms** (Lab 8).